```
case similar([SchemaMatchCandidate])

    /// 无匹配
    case none
}

/// 指纹相似度评分
public struct FingerprintSimilarity: Sendable {
    public let fingerprint: FileFingerprint
    public let score: Double  // 0.0 - 1.0
    public let breakdown: SimilarityBreakdown

    public var isHighConfidence: Bool { score >= 0.9 }
    public var isMediumConfidence: Bool { score >= 0.7 }
}

/// 相似度评分细节
public struct SimilarityBreakdown: Sendable {
    public let sheetNameScore: Double
    public let dimensionScore: Double
    public let headerHashScore: Double
    public let sampleDataScore: Double
}

// File: Sources/xlsOneCore/Schema/SchemaMatcher.swift
import Foundation

/// Schema 匹配器 - 根据文件指纹匹配最合适的 Schema
public struct SchemaMatcher {

    /// 高置信度阈值
    public static let highConfidenceThreshold = 0.9

    /// 中置信度阈值
    public static let mediumConfidenceThreshold = 0.7

    /// 匹配 Schema
    /// - Parameters:
    ///   - fingerprint: 目标文件指纹
    ///   - schemas: 可用的 Schema 列表
    /// - Returns: 匹配结果
    public static func match(
        fingerprint: FileFingerprint,
        against schemas: [MergeSchema]
    ) -> SchemaMatchResult {
        guard !schemas.isEmpty else { return .none }

        // 计算每个 Schema 的相似度
        var similarities: [(schema: MergeSchema, score: Double, breakdown: SimilarityBreakdown)] = []

        for schema in schemas {
            let (score, breakdown) = calculateSimilarity(
                fingerprint,
                schema.fingerprint
            )
            similarities.append((schema, score, breakdown))
        }

        // 按相似度排序
        similarities.sort { $0.score > $1.score }
```

```
// 检查是否有高置信匹配；多个高置信命中时不静默任选
        let exactMatches = similarities.filter { $0.score >= highConfidenceThreshold }
        if exactMatches.count == 1, let exact = exactMatches.first {
            return .exact(exact.schema)
        }
        if exactMatches.count > 1 {
            return .ambiguous(exactMatches.map { SchemaMatchCandidate(schema: $0.schema, score: $0.score) })
        }

        // 返回相似匹配（中置信度及以上）
        let similarMatches = similarities.filter { $0.score >= mediumConfidenceThreshold }
        if !similarMatches.isEmpty {
            let results = similarMatches.map { SchemaMatchCandidate(schema: $0.schema, score: $0.score) }
            return .similar(results)
        }

        return .none
    }

    /// 匹配工作区级规则。一个工作区只能自动应用一套明确规则。
    public static func match(
        workbookFingerprint: WorkbookRuleFingerprint,
        against schemas: [MergeSchema]
    ) -> SchemaMatchResult {
        guard !schemas.isEmpty else { return .none }

        var similarities: [(schema: MergeSchema, score: Double)] = []

        for schema in schemas {
            let score: Double
            if let savedWorkbookFingerprint = schema.workbookFingerprint {
                score = calculateWorkbookSimilarity(
                    workbookFingerprint,
                    savedWorkbookFingerprint
                )
            } else {
                score = calculateLegacySimilarity(
                    workbookFingerprint: workbookFingerprint,
                    schemaFingerprint: schema.fingerprint
                )
            }
            similarities.append((schema, score))
        }

        similarities.sort { lhs, rhs in
            if lhs.score != rhs.score {
                return lhs.score > rhs.score
            }
            return lhs.schema.updatedAt > rhs.schema.updatedAt
        }

        let exactMatches = similarities.filter { $0.score >= highConfidenceThreshold }
        if exactMatches.count == 1, let exact = exactMatches.first {
            return .exact(exact.schema)
        }
        if exactMatches.count > 1 {
            return .ambiguous(exactMatches.map { SchemaMatchCandidate(schema: $0.schema, score: $0.score) })
        }
```

```
let similarMatches = similarities.filter { $0.score >= mediumConfidenceThreshold }
        if !similarMatches.isEmpty {
            return .similar(similarMatches.map { SchemaMatchCandidate(schema: $0.schema, score: $0.score) })
        }

        return .none
    }

    /// 计算两个指纹的相似度
    /// - Returns: (相似度分数 0-1, 评分细节)
    public static func calculateSimilarity(
        _ fp1: FileFingerprint,
        _ fp2: FileFingerprint
    ) -> (score: Double, breakdown: SimilarityBreakdown) {
        var score = 0.0
        var totalWeight = 0.0

        // 1. 工作表名称匹配 (权重: 25%)
        totalWeight += 0.25
        let sheetNameScore = fp1.sheetName == fp2.sheetName ? 1.0 : 0.0
        score += 0.25 * sheetNameScore

        // 2. 行列维度匹配 (权重: 20%)
        // 允许 10% 的误差范围
        totalWeight += 0.20
        let rowRatio = ratioScore(fp1.rowCount, fp2.rowCount, tolerance: 0.1)
        let colRatio = ratioScore(fp1.colCount, fp2.colCount, tolerance: 0.1)
        let dimensionScore = (rowRatio + colRatio) / 2
        score += 0.20 * dimensionScore

        // 3. 表头哈希匹配 (权重: 35%)
        totalWeight += 0.35
        let headerScore = fp1.headerHash == fp2.headerHash ? 1.0 : 0.0
        score += 0.35 * headerScore

        // 4. 样本数据哈希匹配 (权重: 20%)
        totalWeight += 0.20
        let sampleScore = fp1.sampleDataHash == fp2.sampleDataHash ? 1.0 : 0.0
        score += 0.20 * sampleScore

        let breakdown = SimilarityBreakdown(
            sheetNameScore: sheetNameScore,
            dimensionScore: dimensionScore,
            headerHashScore: headerScore,
            sampleDataScore: sampleScore
        )

        // 归一化分数
        return (score / totalWeight, breakdown)
    }

    public static func calculateWorkbookSimilarity(
        _ fp1: WorkbookRuleFingerprint,
        _ fp2: WorkbookRuleFingerprint
    ) -> Double {
        guard !fp1.sheetFingerprints.isEmpty,
              fp1.sheetFingerprints.count == fp2.sheetFingerprints.count else {
            return 0
        }
```

```
var totalScore = 0.0
        var sheetNamesAndDimensionsMatch = true

        for (lhs, rhs) in zip(fp1.sheetFingerprints, fp2.sheetFingerprints) {
            if lhs.sheetName != rhs.sheetName ||
                lhs.rowCount != rhs.rowCount ||
                lhs.columnCount != rhs.columnCount {
                sheetNamesAndDimensionsMatch = false
            }

            var sheetScore = 0.0
            sheetScore += lhs.sheetName == rhs.sheetName ? 0.30 : 0
            sheetScore += (lhs.rowCount == rhs.rowCount && lhs.columnCount == rhs.columnCount) ? 0.30 : 0
            sheetScore += lhs.layoutHash == rhs.layoutHash ? 0.25 : 0
            sheetScore += lhs.formatHash == rhs.formatHash ? 0.15 : 0
            totalScore += sheetScore
        }

        let score = totalScore / Double(fp1.sheetFingerprints.count)
        if sheetNamesAndDimensionsMatch {
            return max(score, highConfidenceThreshold)
        }
        return score
    }

    private static func calculateLegacySimilarity(
        workbookFingerprint: WorkbookRuleFingerprint,
        schemaFingerprint: FileFingerprint
    ) -> Double {
        guard let matchingSheet = workbookFingerprint.sheetFingerprints.first(where: {
            $0.sheetName == schemaFingerprint.sheetName
        }) else {
            return 0
        }

        let dimensionsMatch = matchingSheet.rowCount == schemaFingerprint.rowCount &&
            matchingSheet.columnCount == schemaFingerprint.colCount
        var score = 0.0
        score += matchingSheet.sheetName == schemaFingerprint.sheetName ? 0.30 : 0
        score += dimensionsMatch ? 0.30 : 0
        score += matchingSheet.layoutHash == schemaFingerprint.headerHash ? 0.25 : 0
        score += matchingSheet.formatHash == schemaFingerprint.sampleDataHash ? 0.15 : 0

        if schemaFingerprint.schemaVersion < 2,
           dimensionsMatch {
            let legacyFloor = workbookFingerprint.sheetFingerprints.count == 1
                ? highConfidenceThreshold
                : mediumConfidenceThreshold
            score = max(score, legacyFloor)
        }

        return score
    }

    /// 计算比例相似度
    /// - Parameters:
    ///   - v1: 值 1
    ///   - v2: 值 2
    ///   - tolerance: 允许误差比例
    /// - Returns: 相似度 0-1
```

```
private static func ratioScore(_ v1: Int, _ v2: Int, tolerance: Double) -> Double {
        if v1 == v2 { return 1.0 }
        if v1 == 0 || v2 == 0 { return 0.0 }

        let ratio = Double(min(v1, v2)) / Double(max(v1, v2))
        if ratio >= (1.0 - tolerance) {
            return ratio
        }
        return 0.0
    }

    /// 查找最佳匹配的 Schema（简化接口）
    public static func findBestMatch(
        fingerprint: FileFingerprint,
        schemas: [MergeSchema]
    ) -> MergeSchema? {
        let result = match(fingerprint: fingerprint, against: schemas)

        switch result {
        case .exact(let schema):
            return schema
        case .ambiguous:
            return nil
        case .similar(let matches):
            return matches.first?.schema
        case .none:
            return nil
        }
    }
}

// File: Sources/xlsOneCore/Schema/SchemaRepository.swift
import Foundation

/// Schema 存储仓库协议
public protocol SchemaRepositoryProtocol: Sendable {
    /// 加载所有 Schema
    func loadAllSchemas() async throws -> [MergeSchema]

    /// 保存 Schema
    func saveSchema(_ schema: MergeSchema) async throws

    /// 删除 Schema
    func deleteSchema(id: UUID) async throws

    /// 根据 ID 查找 Schema
    func findSchema(id: UUID) async throws -> MergeSchema?

    /// 根据指纹查找匹配的 Schema
    func findMatchingSchema(fingerprint: FileFingerprint) async throws -> SchemaMatchResult

    /// 根据工作区指纹查找匹配的 Schema
    func findMatchingSchema(workbookFingerprint: WorkbookRuleFingerprint) async throws -> SchemaMatchResult

    /// 更新 Schema 匹配计数
    func incrementMatchCount(id: UUID) async throws

    /// 导出 Schema
    func exportSchema(id: UUID) async throws -> Data
```

```
/// 导入 Schema
    func importSchema(data: Data) async throws -> MergeSchema
}

/// 本地文件系统实现的 Schema 仓库
public actor SchemaRepository: SchemaRepositoryProtocol {

    /// 单例实例
    public static let shared = SchemaRepository()

    /// Schema 存储目录
    private let baseDirectory: URL

    /// Schema 索引文件名
    private let indexFileName = "index.json"

    /// 内存中的缓存
    private var cachedSchemas: [MergeSchema]?

    /// 初始化
    public init(baseDirectory: URL? = nil) {
        if let baseDir = baseDirectory {
            self.baseDirectory = baseDir
        } else {
            // 使用应用支持目录
            let appSupport = FileManager.default.urls(
                for: .applicationSupportDirectory,
                in: .userDomainMask
            ).first!
            self.baseDirectory = appSupport.appendingPathComponent("xlsOne/schemas", isDirectory: true)
        }
    }

    // MARK: - SchemaRepositoryProtocol

    public func loadAllSchemas() async throws -> [MergeSchema] {
        // 返回缓存（如果可用）
        if let cached = cachedSchemas {
            return cached
        }

        // 确保目录存在
        try ensureDirectoryExists()

        // 读取索引文件
        let indexURL = baseDirectory.appendingPathComponent(indexFileName)
        guard FileManager.default.fileExists(atPath: indexURL.path) else {
            cachedSchemas = []
            return []
        }

        let data = try Data(contentsOf: indexURL)
        let ids = try JSONDecoder().decode([UUID].self, from: data)

        // 加载每个 Schema
        var schemas: [MergeSchema] = []
        for id in ids {
            if let schema = try await loadSchema(id: id) {
                schemas.append(schema)
            }
```

```
}

        // 按匹配次数排序（常用优先）
        schemas.sort { $0.updatedAt > $1.updatedAt }

        cachedSchemas = schemas
        return schemas
    }

    public func saveSchema(_ schema: MergeSchema) async throws {
        try ensureDirectoryExists()

        // 保存 Schema 文件
        let schemaURL = schemaFileURL(id: schema.id)
        let data = try JSONEncoder().encode(schema)
        try data.write(to: schemaURL)

        // 更新索引
        var ids = try loadIndex()
        if !ids.contains(schema.id) {
            ids.append(schema.id)
            try saveIndex(ids)
        }

        // 更新缓存
        if cachedSchemas != nil {
            // 移除旧的（如果存在）
            cachedSchemas?.removeAll { $0.id == schema.id }
            // 添加新的
            cachedSchemas?.append(schema)
        }
    }

    public func deleteSchema(id: UUID) async throws {
        // 删除文件
        let schemaURL = schemaFileURL(id: id)
        if FileManager.default.fileExists(atPath: schemaURL.path) {
            try FileManager.default.removeItem(at: schemaURL)
        }

        // 更新索引
        var ids = try loadIndex()
        ids.removeAll { $0 == id }
        try saveIndex(ids)

        // 更新缓存
        cachedSchemas?.removeAll { $0.id == id }
    }

    public func findSchema(id: UUID) async throws -> MergeSchema? {
        // 先查缓存
        if let cached = cachedSchemas?.first(where: { $0.id == id }) {
            return cached
        }

        // 从文件加载
        return try await loadSchema(id: id)
    }

    public func findMatchingSchema(fingerprint: FileFingerprint) async throws -> SchemaMatchResult {
```

```
let schemas = try await loadAllSchemas()
        return SchemaMatcher.match(fingerprint: fingerprint, against: schemas)
    }

    public func findMatchingSchema(workbookFingerprint: WorkbookRuleFingerprint) async throws -> SchemaMatchResult {
        let schemas = try await loadAllSchemas()
        return SchemaMatcher.match(workbookFingerprint: workbookFingerprint, against: schemas)
    }

    public func incrementMatchCount(id: UUID) async throws {
        guard let schema = try await findSchema(id: id) else { return }

        // 创建更新后的 Schema
        let updatedSchema = MergeSchema(
            id: schema.id,
            name: schema.name,
            fingerprint: schema.fingerprint,
            workbookFingerprint: schema.workbookFingerprint,
            cellOverrides: schema.cellOverrides,
            createdAt: schema.createdAt,
            updatedAt: Date()
        )

        try await saveSchema(updatedSchema)
    }

    // MARK: - 导入/导出

    /// 导出 Schema 为 JSON 数据
    public func exportSchema(id: UUID) async throws -> Data {
        guard let schema = try await findSchema(id: id) else {
            throw SchemaRepositoryError.schemaNotFound
        }

        let wrapper = ExportedSchema(
            version: "1.0",
            exportedAt: Date(),
            schema: schema
        )

        return try JSONEncoder().encode(wrapper)
    }

    /// 从 JSON 数据导入 Schema
    public func importSchema(data: Data) async throws -> MergeSchema {
        let newSchema = try Self.decodeImportableSchema(data: data)
        try await saveSchema(newSchema)
        return newSchema
    }

    /// 解析可导入的调整记忆，但不写入本地仓库。
    public nonisolated static func decodeImportableSchema(data: Data) throws -> MergeSchema {
        let decoder = JSONDecoder()

        if let wrapper = try? decoder.decode(ExportedSchema.self, from: data) {
            return importedCopy(of: wrapper.schema)
        }

        let schema = try decoder.decode(MergeSchema.self, from: data)
        return importedCopy(of: schema)
```

```
}

    // MARK: - 私有方法

    private func ensureDirectoryExists() throws {
        if !FileManager.default.fileExists(atPath: baseDirectory.path) {
            try FileManager.default.createDirectory(
                at: baseDirectory,
                withIntermediateDirectories: true
            )
        }
    }

    private func schemaFileURL(id: UUID) -> URL {
        baseDirectory.appendingPathComponent("schema-\(id.uuidString).json")
    }

    private func loadSchema(id: UUID) async throws -> MergeSchema? {
        let url = schemaFileURL(id: id)
        guard FileManager.default.fileExists(atPath: url.path) else { return nil }

        let data = try Data(contentsOf: url)
        return try JSONDecoder().decode(MergeSchema.self, from: data)
    }

    private func loadIndex() throws -> [UUID] {
        let url = baseDirectory.appendingPathComponent(indexFileName)
        guard FileManager.default.fileExists(atPath: url.path) else { return [] }

        let data = try Data(contentsOf: url)
        return try JSONDecoder().decode([UUID].self, from: data)
    }

    private func saveIndex(_ ids: [UUID]) throws {
        let url = baseDirectory.appendingPathComponent(indexFileName)
        let data = try JSONEncoder().encode(ids)
        try data.write(to: url)
    }

    private nonisolated static func importedCopy(of schema: MergeSchema) -> MergeSchema {
        MergeSchema(
            id: UUID(),
            name: schema.name + " (导入)",
            fingerprint: schema.fingerprint,
            workbookFingerprint: schema.workbookFingerprint,
            cellOverrides: schema.cellOverrides
        )
    }
}

/// 导出的 Schema 包装器
private struct ExportedSchema: Codable {
    let version: String
    let exportedAt: Date
    let schema: MergeSchema
}

/// 仓库错误
public enum SchemaRepositoryError: Error {
    case schemaNotFound
```

```
case invalidData
    case saveFailed
}

// File: Sources/xlsOneCore/Schema/FingerprintGenerator.swift
import Foundation
import CryptoKit

/// 文件指纹生成器
public struct FingerprintGenerator {

    /// 从 ExcelFile 生成指纹
    /// - Parameters:
    ///   - file: Excel 文件
    ///   - fileNamePattern: 可选的文件名匹配模式
    /// - Returns: 文件指纹
    public static func generate(from file: ExcelFile, fileNamePattern: String? = nil) -> FileFingerprint {
        // 使用第一个工作表作为主要特征源
        guard let sheet = file.sheets.first else {
            return FileFingerprint(
                sheetName: "",
                rowCount: 0,
                colCount: 0,
                headerHash: "",
                sampleDataHash: "",
                fileNamePattern: fileNamePattern
            )
        }

        let headerHash = generateHeaderHash(sheet: sheet)
        let sampleDataHash = generateSampleDataHash(sheet: sheet)
        let maxCols = sheet.rows.map { $0.count }.max() ?? 0

        return FileFingerprint(
            sheetName: sheet.name,
            rowCount: sheet.rows.count,
            colCount: maxCols,
            headerHash: headerHash,
            sampleDataHash: sampleDataHash,
            fileNamePattern: fileNamePattern
        )
    }

    /// 从指定工作表生成兼容旧规则的单表指纹
    public static func generate(
        from file: ExcelFile,
        sheetName: String,
        fileNamePattern: String? = nil
    ) -> FileFingerprint {
        guard let sheet = file.sheets.first(where: { $0.name == sheetName }) else {
            return FileFingerprint(
                sheetName: sheetName,
                rowCount: 0,
                colCount: 0,
                headerHash: "",
                sampleDataHash: "",
                fileNamePattern: fileNamePattern
            )
        }
```

```
let dimensions = effectiveDimensions(for: sheet)
        let sheetFingerprint = generateSheetRuleFingerprint(
            sheet: sheet,
            effectiveRowCount: dimensions.rows,
            effectiveColumnCount: dimensions.cols
        )

        return FileFingerprint(
            schemaVersion: 2,
            sheetName: sheet.name,
            rowCount: sheetFingerprint.rowCount,
            colCount: sheetFingerprint.columnCount,
            headerHash: sheetFingerprint.layoutHash,
            sampleDataHash: sheetFingerprint.formatHash,
            fileNamePattern: fileNamePattern
        )
    }

    /// 从一组同构 Excel 的可合并工作表生成工作区级规则指纹
    public static func generateWorkbook(
        from files: [ExcelFile],
        sheetNames: [String]
    ) -> WorkbookRuleFingerprint {
        let fingerprints = sheetNames.map { sheetName in
            generateConsensusSheetFingerprint(from: files, sheetName: sheetName)
        }

        return WorkbookRuleFingerprint(sheetFingerprints: fingerprints)
    }

    /// 为新规则提供兼容旧字段的代表性指纹
    public static func generateWorkspaceLegacyFingerprint(
        from files: [ExcelFile],
        sheetNames: [String]
    ) -> FileFingerprint {
        guard let firstSheetName = sheetNames.first,
              let file = files.first else {
            return FileFingerprint(
                schemaVersion: 2,
                sheetName: "",
                rowCount: 0,
                colCount: 0,
                headerHash: "",
                sampleDataHash: ""
            )
        }

        return generate(from: file, sheetName: firstSheetName)
    }

    /// 生成表头哈希
    /// 结合工作表名称、第一行（列头）和第一列（行标签）
    private static func generateHeaderHash(sheet: SheetData) -> String {
        var components: [String] = []

        // 添加工作表名称
        components.append(sheet.name)

        // 添加第一行内容（列头）- 最多前 10 列
        if let firstRow = sheet.rows.first {
```

```
let limit = min(firstRow.count, 10)
            for i in 0..<limit {
                let value = firstRow[i].value.trimmingCharacters(in: .whitespaces)
                // 标准化：移除空格、转为小写
                components.append(normalize(value))
            }
        }

        // 添加第一列内容（行标签）- 最多前 20 行
        let rowLimit = min(sheet.rows.count, 20)
        for i in 0..<rowLimit {
            if let firstCell = sheet.rows[i].first {
                let value = firstCell.value.trimmingCharacters(in: .whitespaces)
                components.append(normalize(value))
            }
        }

        return hash(components.joined(separator: "|"))
    }

    /// 生成样本数据哈希
    /// 选取表格四角和中心点的单元格值
    private static func generateSampleDataHash(sheet: SheetData) -> String {
        let maxRows = sheet.rows.count
        guard maxRows > 0 else { return "" }

        let maxCols = sheet.rows.map { $0.count }.max() ?? 0
        guard maxCols > 0 else { return "" }

        // 选取关键点
        var samplePoints: [(row: Int, col: Int)] = [
            (0, 0),                                  // 左上
            (0, max(0, maxCols - 1)),                // 右上
            (max(0, maxRows - 1), 0),                // 左下
            (max(0, maxRows - 1), max(0, maxCols - 1)) // 右下
        ]

        // 添加中心点
        if maxRows > 2 && maxCols > 2 {
            samplePoints.append((maxRows / 2, maxCols / 2))
        }

        // 收集样本值
        var sampleValues: [String] = []
        for point in samplePoints {
            if point.row < maxRows,
               point.col < sheet.rows[point.row].count {
                let value = sheet.rows[point.row][point.col].value
                sampleValues.append(normalize(value))
            }
        }

        return hash(sampleValues.joined(separator: "|"))
    }

    /// 标准化字符串（用于更稳定的匹配）
    private static func normalize(_ text: String) -> String {
        return text
            .lowercased()
            .trimmingCharacters(in: .whitespaces)
```

```
.replacingOccurrences(of: "\\s+", with: "", options: .regularExpression)
    }

    /// 计算字符串哈希（使用 SHA256 的前 16 位）
    private static func hash(_ text: String) -> String {
        let data = Data(text.utf8)
        let hash = SHA256.hash(data: data)
        // 取前 8 字节（16 个十六进制字符）作为哈希值
        return hash.prefix(8).map { String(format: "%02x", $0) }.joined()
    }

    private static func generateConsensusSheetFingerprint(
        from files: [ExcelFile],
        sheetName: String
    ) -> SheetRuleFingerprint {
        let sheets = files.compactMap { file in
            file.sheets.first(where: { $0.name == sheetName })
        }

        let dimensions = chooseDominantDimensions(
            from: sheets.map(effectiveDimensions)
        )

        guard let representativeSheet = sheets.first else {
            return SheetRuleFingerprint(
                sheetName: sheetName,
                rowCount: 0,
                columnCount: 0,
                layoutHash: "",
                formatHash: ""
            )
        }

        return generateSheetRuleFingerprint(
            sheet: representativeSheet,
            effectiveRowCount: dimensions.rows,
            effectiveColumnCount: dimensions.cols
        )
    }

    private static func generateSheetRuleFingerprint(
        sheet: SheetData,
        effectiveRowCount: Int,
        effectiveColumnCount: Int
    ) -> SheetRuleFingerprint {
        var layoutComponents: [String] = [sheet.name, "\(effectiveRowCount)x\(effectiveColumnCount)"]
        var formatComponents: [String] = []

        for rowIndex in 0..<effectiveRowCount {
            for columnIndex in 0..<effectiveColumnCount {
                let cell = sheet.cellAt(row: rowIndex, col: columnIndex)
                let isEmpty = cell?.value.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty ?? true
                layoutComponents.append(isEmpty ? "0" : "1")
                formatComponents.append(structuralToken(for: cell))
            }
        }

        return SheetRuleFingerprint(
            sheetName: sheet.name,
            rowCount: effectiveRowCount,
```

```
columnCount: effectiveColumnCount,
            layoutHash: hash(layoutComponents.joined(separator: "|")),
            formatHash: hash(formatComponents.joined(separator: "|"))
        )
    }

    private static func structuralToken(for cell: CellData?) -> String {
        guard let cell else { return "E" }
        let value = cell.value.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !value.isEmpty else { return "E" }
        if cell.isDate { return "D" }
        if cell.numericValue != nil { return "N" }
        if value.range(of: #"\p{Han}"#, options: .regularExpression) != nil { return "C" }
        if value.range(of: #"[A-Za-z]"#, options: .regularExpression) != nil { return "A" }
        return "T"
    }

    private static func effectiveDimensions(for sheet: SheetData) -> SheetDimensions {
        var lastNonEmptyRow = -1
        var lastNonEmptyColumn = -1

        for (rowIndex, row) in sheet.rows.enumerated() {
            for (columnIndex, cell) in row.enumerated()
            where !cell.value.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
                lastNonEmptyRow = max(lastNonEmptyRow, rowIndex)
                lastNonEmptyColumn = max(lastNonEmptyColumn, columnIndex)
            }
        }

        return SheetDimensions(
            rows: max(lastNonEmptyRow + 1, 0),
            cols: max(lastNonEmptyColumn + 1, 0)
        )
    }

    private static func chooseDominantDimensions(from dimensions: [SheetDimensions]) -> SheetDimensions {
        guard !dimensions.isEmpty else { return SheetDimensions(rows: 0, cols: 0) }

        return Dictionary(grouping: dimensions.enumerated(), by: \.element)
            .values
            .max { lhs, rhs in
                if lhs.count != rhs.count {
                    return lhs.count < rhs.count
                }

                let lhsDimensions = lhs[0].element
                let rhsDimensions = rhs[0].element
                if lhsDimensions != rhsDimensions {
                    return lhsDimensions < rhsDimensions
                }

                let lhsFirstIndex = lhs.map(\.offset).min() ?? .max
                let rhsFirstIndex = rhs.map(\.offset).min() ?? .max
                return lhsFirstIndex > rhsFirstIndex
            }?
            .first?
            .element ?? SheetDimensions(rows: 0, cols: 0)
    }
}
```

```
private struct SheetDimensions: Hashable, Comparable {
    let rows: Int
    let cols: Int

    static func < (lhs: SheetDimensions, rhs: SheetDimensions) -> Bool {
        if lhs.rows != rhs.rows {
            return lhs.rows < rhs.rows
        }
        return lhs.cols < rhs.cols
    }
}

// File: Sources/xlsOneLicense/DeviceFingerprint.swift
import Foundation
import CryptoKit

/// Generates a stable device fingerprint for license binding.
/// Uses a Keychain-stored UUID — survives reboots, survives Keychain reset
/// only if the user explicitly removes the entry.
enum DeviceFingerprint {

    static func generate() -> String {
        // Primary: Keychain-stored UUID (App Store compatible, no IOKit needed)
        if let existing = Keychain.load(key: "com.xlsone.deviceUUID") {
            return sha256(existing + ".xlsone.device")
        }

        // First launch: generate and persist a new UUID
        let uuid = UUID().uuidString
        _ = Keychain.save(key: "com.xlsone.deviceUUID", data: uuid)
        return sha256(uuid + ".xlsone.device")
    }

    // MARK: - Private

    private static func sha256(_ input: String) -> String {
        let data = Data(input.utf8)
        let hash = SHA256.hash(data: data)
        return hash.compactMap { String(format: "%02x", $0) }.joined()
    }
}

// File: Sources/xlsOneLicense/Keychain.swift
import Foundation
import Security

/// Minimal Keychain wrapper for storing license tokens and device identifiers.
enum Keychain {

    static func save(key: String, data: String) -> Bool {
        delete(key: key)

        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecAttrService as String: "com.xlsone.license",
            kSecValueData as String: Data(data.utf8),
            kSecAttrAccessible as String: kSecAttrAccessibleAfterFirstUnlock,
        ]
```

```
let status = SecItemAdd(query as CFDictionary, nil)
        return status == errSecSuccess
    }

    static func load(key: String) -> String? {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecAttrService as String: "com.xlsone.license",
            kSecReturnData as String: true,
            kSecMatchLimit as String: kSecMatchLimitOne,
        ]

        var item: CFTypeRef?
        let status = SecItemCopyMatching(query as CFDictionary, &item)
        guard status == errSecSuccess, let data = item as? Data else { return nil }

        return String(data: data, encoding: .utf8)
    }

    static func delete(key: String) {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecAttrService as String: "com.xlsone.license",
        ]
        SecItemDelete(query as CFDictionary)
    }
}

// File: Sources/xlsOneLicense/LicenseManager.swift
import Foundation
import xlsOneCore
#if os(macOS)
import AppKit
#endif

// MARK: - License Manager

/// Manages software license activation, verification, and offline support.
/// Communicates with the xlsOne activation API for online validation.
public final class LicenseManager: ObservableObject {

    public static let shared = LicenseManager()
    public static var isAppStoreDistribution: Bool {
        Bundle.main.object(forInfoDictionaryKey: "XLSONEDistributionChannel") as? String == "app-store"
    }

    // MARK: - Published State

    @Published public private(set) var licenseState: LicenseState = .unactivated
    @Published public private(set) var plan: LicensePlan = .unknown
    @Published public private(set) var isVerifying = false
    @Published public var showActivationSheet = false

    // MARK: - Constants

    private enum API {
        /// Primary endpoint (Cloudflare Workers — global)
        static let primary = "https://api.xlsone.com"
```

```
/// Fallback endpoint (Aliyun FC — China mainland)
        static let fallback = "https://"
        /// Request timeout
        static let timeout: TimeInterval = 3.0
    }

    private enum Storage {
        static let tokenKey = "com.xlsone.license.token"
        static let deviceIDKey = "com.xlsone.license.deviceID"
        static let offlineLicenseKey = "com.xlsone.license.offline"
        static let trialStartKey = "com.xlsone.license.trialStart"
    }

    private static let trialDurationDays = 14

    // MARK: - Initialization

    private init() {
        if Self.isAppStoreDistribution {
            licenseState = .activated
            plan = .appStore
        } else {
            loadPersistedState()
        }
    }

    // MARK: - Public API

    /// Start a free trial. Returns the remaining days.
    public func startTrial() -> Int {
        if Self.isAppStoreDistribution {
            licenseState = .activated
            plan = .appStore
            return Int.max
        }

        let now = Date()
        UserDefaults.standard.set(now, forKey: Storage.trialStartKey)
        let remaining = Self.trialDurationDays
        licenseState = .trial(remainingDays: remaining)
        return remaining
    }

    /// Check trial status. Returns remaining days or -1 if trial expired/not started.
    public func checkTrialStatus() -> Int {
        guard let start = UserDefaults.standard.object(forKey: Storage.trialStartKey) as? Date else {
            return -1
        }
        let elapsed = Calendar.current.dateComponents([.day], from: start, to: Date()).day ?? 0
        let remaining = Self.trialDurationDays - elapsed
        if remaining > 0 {
            return remaining
        }
        return -1
    }

    /// Import an offline license file (JWT token).
    @discardableResult
    @MainActor
    public func importOfflineLicenseFile() -> ActivationResult? {
```

```
guard !Self.isAppStoreDistribution else {
            return .success(plan: .appStore, expiresAt: nil)
        }

#if os(macOS)
        let panel = NSOpenPanel()
        panel.allowedContentTypes = [.data, .json]
        panel.allowsMultipleSelection = false
        panel.message = "选择授权文件"

        guard Self.runCenteredModal(panel) == .OK, let url = panel.url else { return nil }

        guard let token = try? String(contentsOf: url, encoding: .utf8).trimmingCharacters(in: .whitespacesAndNewlines),
              !token.isEmpty
        else {
            DispatchQueue.main.async {
                self.licenseState = .unactivated
            }
            return .failure(.invalidLicenseFile)
        }

        let deviceID = getOrCreateDeviceID()
        guard let payload = decodeTokenPayload(token) else {
            DispatchQueue.main.async {
                self.licenseState = .unactivated
            }
            return .failure(.invalidLicenseFile)
        }

        guard payload.dev == deviceID else {
            return .failure(.licenseDeviceMismatch)
        }

        guard isTokenValidLocally(payload, deviceID: deviceID) else {
            return .failure(.invalidLicenseFile)
        }

        let plan = LicensePlan(rawValue: payload.plan) ?? .unknown
        saveToken(token)
        UserDefaults.standard.set(token, forKey: Storage.offlineLicenseKey)
        DispatchQueue.main.async {
            self.licenseState = .activated
            self.plan = plan
        }
        return .success(plan: plan, expiresAt: nil)
#else
        return .failure(.serverError(LocaleManager.loc("当前平台不支持导入授权文件")))
#endif
    }

    /// Activate with an activation key. Returns the result.
    public func activate(key: String) async -> ActivationResult {
        if Self.isAppStoreDistribution {
            await MainActor.run {
                self.licenseState = .activated
                self.plan = .appStore
            }
            return .success(plan: .appStore, expiresAt: nil)
        }
```

```
let deviceID = getOrCreateDeviceID()
        let normalizedKey = key.uppercased().trimmingCharacters(in: .whitespaces)

        guard KeyFormat.isValid(normalizedKey) else {
            return .failure(.invalidKeyFormat)
        }

        let body: [String: String] = [
            "key": normalizedKey,
            "device_id": deviceID,
            "device_name": Host.current().localizedName ?? "Unknown Mac"
        ]

        // Try primary endpoint first
        if let result = await postActivation(to: API.primary, body: body) {
            return result
        }

        // Fallback to China endpoint
        if let result = await postActivation(to: API.fallback, body: body) {
            return result
        }

        // Allow offline activation attempt from stored offline license
        if let offlineResult = tryOfflineActivation(key: normalizedKey, deviceID: deviceID) {
            return offlineResult
        }

        return .failure(.networkError)
    }

    /// Verify current license validity. Called on app launch.
    public func verifyOnLaunch() async {
        if Self.isAppStoreDistribution {
            await MainActor.run {
                self.licenseState = .activated
                self.plan = .appStore
                self.showActivationSheet = false
            }
            return
        }

        await MainActor.run { isVerifying = true }
        defer { Task { @MainActor in isVerifying = false } }

        guard let token = loadToken() else {
            await setState(.unactivated)
            return
        }

        let deviceID = getOrCreateDeviceID()

        // Try online verification
        if let result = await postVerify(to: API.primary, token: token, deviceID: deviceID) {
            await handleVerifyResult(result)
            return
        }

        // Fallback endpoint
        if let result = await postVerify(to: API.fallback, token: token, deviceID: deviceID) {
```

```
await handleVerifyResult(result)
            return
        }

        // Offline validation — check token locally
        if let payload = decodeTokenPayload(token), isTokenValidLocally(payload, deviceID: deviceID) {
            await setState(.activated)
            await setPlan(LicensePlan(rawValue: payload.plan) ?? .unknown)
            return
        }

        // Token invalid, try offline license file
        if let offlineToken = loadOfflineLicense(), let payload = decodeTokenPayload(offlineToken),
           isTokenValidLocally(payload, deviceID: deviceID) {
            await setState(.activated)
            await setPlan(LicensePlan(rawValue: payload.plan) ?? .unknown)
            return
        }

        // Grace period: if token was valid recently (within 7 days), allow
        if let lastValid = UserDefaults.standard.object(forKey: "com.xlsone.license.lastValid") as? Date,
           Date().timeIntervalSince(lastValid) < 7 * 24 * 60 * 60 {
            await setState(.gracePeriod)
            return
        }

        await setState(.expired)
    }

    /// Refresh subscription token. Called periodically.
    public func refreshIfNeeded() async {
        guard !Self.isAppStoreDistribution else { return }

        guard case .activated = licenseState,
              let token = loadToken(),
              let payload = decodeTokenPayload(token),
              payload.exp > 0, // lifetime licenses don't need refresh
              payload.exp < Int(Date().timeIntervalSince1970) + 7 * 24 * 60 * 60 // refresh if expiring within 7 days
        else { return }

        let deviceID = getOrCreateDeviceID()

        let body: [String: String] = ["token": token, "device_id": deviceID]

        if let data = await postJSON(to: "\(API.primary)/api/refresh", body: body),
           let result = try? JSONDecoder().decode(RefreshResponse.self, from: data),
           result.valid {
            if let token = result.token { saveToken(token) }
        }
    }

    /// Reset license (for user logout / key change)
    public func reset() {
        guard !Self.isAppStoreDistribution else {
            licenseState = .activated
            plan = .appStore
            showActivationSheet = false
            return
        }
```

```
Keychain.delete(key: Storage.tokenKey)
        UserDefaults.standard.removeObject(forKey: Storage.offlineLicenseKey)
        licenseState = .unactivated
        plan = .unknown
    }

    // MARK: - Private — Network

    private func postActivation(to base: String, body: [String: String]) async -> ActivationResult? {
        guard let data = await postJSON(to: "\(base)/api/activate", body: body) else {
            return nil
        }

        guard let result = try? JSONDecoder().decode(ActivateResponse.self, from: data) else {
            if let error = try? JSONDecoder().decode(APIError.self, from: data) {
                return .failure(mapAPIError(error))
            }
            return nil
        }

        saveToken(result.license_token)

        let plan = LicensePlan(rawValue: result.plan) ?? .personalYearly
        Task { @MainActor in
            self.licenseState = .activated
            self.plan = plan
            UserDefaults.standard.set(Date(), forKey: "com.xlsone.license.lastValid")
        }

        return .success(plan: plan, expiresAt: result.expires_at)
    }

    private func postVerify(to base: String, token: String, deviceID: String) async -> VerifyResponse? {
        let body: [String: String] = ["token": token, "device_id": deviceID]
        guard let data = await postJSON(to: "\(base)/api/verify", body: body) else {
            return nil
        }
        return try? JSONDecoder().decode(VerifyResponse.self, from: data)
    }

    private func handleVerifyResult(_ result: VerifyResponse) async {
        if result.valid {
            await setState(.activated)
            await setPlan(LicensePlan(rawValue: result.plan) ?? .unknown)
            UserDefaults.standard.set(Date(), forKey: "com.xlsone.license.lastValid")
        } else if result.expired == true {
            await setState(.expired)
        } else {
            await setState(.unactivated)
        }
    }

    private func postJSON(to urlString: String, body: [String: String]) async -> Data? {
        guard let url = URL(string: urlString) else { return nil }

        var request = URLRequest(url: url, timeoutInterval: API.timeout)
        request.httpMethod = "POST"
        request.setValue("application/json", forHTTPHeaderField: "Content-Type")
        request.httpBody = try? JSONSerialization.data(withJSONObject: body)
```

```
do {
            let (data, response) = try await URLSession.shared.data(for: request)
            guard let http = response as? HTTPURLResponse, (200...299).contains(http.statusCode) else {
                return nil
            }
            return data
        } catch {
            return nil
        }
    }

    // MARK: - Private — Offline

    private func tryOfflineActivation(key: String, deviceID: String) -> ActivationResult? {
        // Offline activation requires a pre-generated .license file
        guard let licenseData = loadOfflineLicense() else { return nil }

        // Verify the offline license matches the key and device
        guard let payload = decodeTokenPayload(licenseData),
              payload.sub == key,
              payload.dev == deviceID,
              isTokenValidLocally(payload, deviceID: deviceID)
        else { return nil }

        saveToken(licenseData)
        let plan = LicensePlan(rawValue: payload.plan) ?? .personalLifetime
        Task { @MainActor in
            self.licenseState = .activated
            self.plan = plan
        }
        return .success(plan: plan, expiresAt: payload.exp > 0
            ? Date(timeIntervalSince1970: TimeInterval(payload.exp)).ISO8601Format() : nil)
    }

    private func isTokenValidLocally(_ payload: TokenPayload, deviceID: String) -> Bool {
        guard payload.dev == deviceID else { return false }
        // Lifetime token
        if payload.exp == 0 { return true }
        // Subscription token with expiry
        return payload.exp > Int(Date().timeIntervalSince1970)
    }

    // MARK: - Private — Storage

    private func loadPersistedState() {
        if let token = loadToken(), let payload = decodeTokenPayload(token),
           isTokenValidLocally(payload, deviceID: getOrCreateDeviceID()) {
            licenseState = .activated
            plan = LicensePlan(rawValue: payload.plan) ?? .unknown
            return
        }
        let remaining = checkTrialStatus()
        if remaining > 0 {
            licenseState = .trial(remainingDays: remaining)
        }
    }

    private func getOrCreateDeviceID() -> String {
        if let existing = Keychain.load(key: Storage.deviceIDKey) {
            return existing
```

```
}
        let id = DeviceFingerprint.generate()
        _ = Keychain.save(key: Storage.deviceIDKey, data: id)
        return id
    }

    private func loadToken() -> String? {
        Keychain.load(key: Storage.tokenKey)
    }

    private func saveToken(_ token: String) {
        _ = Keychain.save(key: Storage.tokenKey, data: token)
    }

    private func loadOfflineLicense() -> String? {
        UserDefaults.standard.string(forKey: Storage.offlineLicenseKey)
    }

    private func decodeTokenPayload(_ token: String) -> TokenPayload? {
        let parts = token.split(separator: ".")
        guard parts.count >= 2 else { return nil }
        // Base64URL decode the payload (second segment)
        var base64 = String(parts[1])
            .replacingOccurrences(of: "-", with: "+")
            .replacingOccurrences(of: "_", with: "/")
        while base64.count % 4 != 0 { base64 += "=" }
        guard let data = Data(base64Encoded: base64),
              let payload = try? JSONDecoder().decode(TokenPayload.self, from: data)
        else { return nil }
        return payload
    }

    @MainActor private func setState(_ state: LicenseState) { self.licenseState = state }
    @MainActor private func setPlan(_ plan: LicensePlan) { self.plan = plan }

#if os(macOS)
    @MainActor
    private static func runCenteredModal(_ panel: NSSavePanel) -> NSApplication.ModalResponse {
        panel.contentView?.layoutSubtreeIfNeeded()
        if let owner = NSApp.keyWindow ?? NSApp.mainWindow ?? NSApp.windows.first(where: { $0.isVisible }) {
            let ownerFrame = owner.frame
            let panelFrame = panel.frame
            panel.setFrameOrigin(NSPoint(
                x: ownerFrame.midX - panelFrame.width / 2,
                y: ownerFrame.midY - panelFrame.height / 2
            ))
        } else {
            panel.center()
        }
        return panel.runModal()
    }
#endif

    private func mapAPIError(_ error: APIError) -> ActivationError {
        switch error.error {
        case "KEY_NOT_FOUND":   return .keyNotFound
        case "KEY_REVOKED":     return .keyRevoked
        case "DEVICE_LIMIT":    return .deviceLimit
        case "RATE_LIMITED":    return .rateLimited
        default:                return .serverError(error.message ?? LocaleManager.loc("未知错误"))
```

```
}
    }
}

// MARK: - Types

public enum LicenseState: Equatable {
    case unactivated
    case activated
    case expired
    case gracePeriod     // Offline, token not verified recently but within grace window
    case trial(remainingDays: Int)  // Free trial with N days remaining
}

public enum LicensePlan: String {
    case appStore = "app_store"
    case personalYearly = "personal_yearly"
    case personalLifetime = "personal_lifetime"
    case enterprise10 = "enterprise_10"
    case enterprise25 = "enterprise_25"
    case enterpriseUnlimited = "enterprise_unlimited"
    case unknown = "unknown"

    public var displayName: String {
        switch self {
        case .appStore:           return LocaleManager.loc("App Store")
        case .personalYearly:      return LocaleManager.loc("个人年度订阅")
        case .personalLifetime:    return LocaleManager.loc("个人永久授权")
        case .enterprise10:        return LocaleManager.loc("企业版 (10 台)")
        case .enterprise25:        return LocaleManager.loc("企业版 (25 台)")
        case .enterpriseUnlimited: return LocaleManager.loc("企业版 (无限)")
        case .unknown:             return LocaleManager.loc("未知")
        }
    }
}

public enum ActivationResult: Equatable {
    case success(plan: LicensePlan, expiresAt: String?)
    case failure(ActivationError)
}

public enum ActivationError: Error, Equatable {
    case invalidKeyFormat
    case invalidLicenseFile
    case licenseDeviceMismatch
    case keyNotFound
    case keyRevoked
    case deviceLimit
    case networkError
    case rateLimited
    case serverError(String)

    public var localizedDescription: String {
        switch self {
        case .invalidKeyFormat: return LocaleManager.loc("激活码格式不正确")
        case .invalidLicenseFile: return LocaleManager.loc("授权文件无效或已过期")
        case .licenseDeviceMismatch: return LocaleManager.loc("授权文件与当前设备不匹配")
        case .keyNotFound:      return LocaleManager.loc("激活码不存在")
        case .keyRevoked:       return LocaleManager.loc("激活码已被吊销")
        case .deviceLimit:      return LocaleManager.loc("已达到最大设备数限制")
```

```
case .networkError:     return LocaleManager.loc("无法连接激活服务器，请检查网络")
        case .rateLimited:      return LocaleManager.loc("请求过于频繁，请稍后再试")
        case .serverError(let m): return LocaleManager.loc("服务器错误: \(m)")
        }
    }
}

// MARK: - API Response Types (internal)

struct ActivateResponse: Decodable {
    let license_token: String
    let plan: String
    let expires_at: String?
    let device_id: String
}

struct VerifyResponse: Decodable {
    let valid: Bool
    let plan: String
    let expires_at: String?
    let expired: Bool?

    enum CodingKeys: String, CodingKey {
        case valid, plan, expires_at
    }

    init(from decoder: Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)
        valid = try container.decode(Bool.self, forKey: .valid)
        plan = (try? container.decode(String.self, forKey: .plan)) ?? "unknown"
        expires_at = try? container.decodeIfPresent(String.self, forKey: .expires_at)
        expired = nil // derived from error code if valid=false
    }
}

struct RefreshResponse: Decodable {
    let valid: Bool
    let token: String?
    let refreshed: Bool?
}

struct APIError: Decodable {
    let error: String
    let message: String?
}

struct TokenPayload: Decodable {
    let sub: String   // key_id
    let dev: String   // device_id
    let plan: String
    let iat: Int
    let exp: Int      // 0 = lifetime
}

// MARK: - Key Format Validation

enum KeyFormat {
    static let pattern = try! NSRegularExpression(pattern: "^[A-Z0-9]{4}-[A-Z0-9]{4}-[A-Z0-9]{4}$")

    static func isValid(_ key: String) -> Bool {
```

```
pattern.firstMatch(in: key, range: NSRange(key.startIndex..., in: key)) != nil
    }
}

// File: Sources/xlsOneLicense/LicenseActivationView.swift
import SwiftUI
import xlsOneCore

// MARK: - License Activation View

/// Modal sheet shown when the app needs activation.
public struct LicenseActivationView: View {
    @ObservedObject var licenseManager: LicenseManager
    @ObservedObject private var localeManager = LocaleManager.shared
    @State private var keyInput = ""
    @State private var isActivating = false
    @State private var errorMessage: String?
    @State private var successMessage: String?
    @State private var showOfflineInfo = false

    public init(licenseManager: LicenseManager = .shared) {
        self.licenseManager = licenseManager
    }

    public var body: some View {
        VStack(spacing: 0) {
            // Icon + Title
            VStack(spacing: 12) {
                Image(systemName: "key.fill")
                    .font(.system(size: 40))
                    .foregroundColor(.accentColor)

                Text(LocaleManager.loc("激活 表表归一"))
                    .font(.title2)
                    .fontWeight(.semibold)

                if licenseManager.licenseState == .expired {
                    Text(LocaleManager.loc("您的许可证已过期，请续费获取新的激活码"))
                        .font(.subheadline)
                        .foregroundColor(.secondary)
                        .multilineTextAlignment(.center)
                } else {
                    Text(LocaleManager.loc("输入激活码以解锁全部功能"))
                        .font(.subheadline)
                        .foregroundColor(.secondary)
                }
            }
            .padding(.top, 32)
            .padding(.bottom, 24)

            // Activation Key Input
            VStack(alignment: .leading, spacing: 8) {
                Text(LocaleManager.loc("激活码"))
                    .font(.callout)
                    .fontWeight(.medium)

                TextField("XXXX-XXXX-XXXX", text: $keyInput)
                    .textFieldStyle(.plain)
                    .font(.system(.body, design: .monospaced))
                    .padding(12)
```

```
.background(Color(nsColor: .controlBackgroundColor))
                    .clipShape(RoundedRectangle(cornerRadius: 8))
                    .overlay(
                        RoundedRectangle(cornerRadius: 8)
                            .stroke(Color.secondary.opacity(0.3), lineWidth: 1)
                    )
                    .onChange(of: keyInput) { newValue in
                        // Auto-insert dashes and uppercase
                        var cleaned = newValue.uppercased().replacingOccurrences(of: "-", with: "")
                        if cleaned.count > 12 { cleaned = String(cleaned.prefix(12)) }

                        var formatted = ""
                        for (i, ch) in cleaned.enumerated() {
                            if i > 0 && i % 4 == 0 { formatted += "-" }
                            formatted.append(ch)
                        }
                        if formatted != newValue {
                            keyInput = formatted
                        }
                    }
            }

            // Error message
            if let error = errorMessage {
                HStack(spacing: 6) {
                    Image(systemName: "exclamationmark.triangle.fill")
                        .font(.caption)
                    Text(error)
                        .font(.caption)
                }
                .foregroundColor(.red)
                .padding(.top, 8)
            }

            if let success = successMessage {
                HStack(spacing: 6) {
                    Image(systemName: "checkmark.circle.fill")
                        .font(.caption)
                    Text(success)
                        .font(.caption)
                }
                .foregroundColor(.green)
                .padding(.top, 8)
            }

            // Spacer
            Spacer().frame(height: 20)

            // Activate Button
            Button(action: activate) {
                HStack(spacing: 8) {
                    if isActivating {
                        ProgressView()
                            .scaleEffect(0.8)
                    }
                    Text(isActivating ? LocaleManager.loc("验证中...") : LocaleManager.loc("激活"))
                        .fontWeight(.semibold)
                }
                .frame(maxWidth: .infinity)
                .padding(.vertical, 12)
```

```
.background(validInput ? Color.accentColor : Color.gray.opacity(0.3))
                .foregroundColor(validInput ? .white : .secondary)
                .clipShape(RoundedRectangle(cornerRadius: 8))
            }
            .disabled(!validInput || isActivating)
            .buttonStyle(.plain)

            Spacer().frame(height: 12)

            // Trial & Purchase
            HStack(spacing: 24) {
                Button(LocaleManager.loc("免费试用 14 天")) {
                    let remaining = licenseManager.startTrial()
                    if remaining > 0 {
                        errorMessage = nil
                        successMessage = nil
                        licenseManager.showActivationSheet = false
                    }
                }
                .buttonStyle(.link)

                Button(LocaleManager.loc("购买激活码 →")) {
                    if let url = URL(string: "https://z-pulse.cn") {
                        NSWorkspace.shared.open(url)
                    }
                }
                .buttonStyle(.link)
            }
            .font(.subheadline)

            Spacer().frame(height: 12)

            // Offline activation
            Button {
                showOfflineInfo.toggle()
            } label: {
                HStack(spacing: 4) {
                    Image(systemName: "wifi.slash")
                    Text(LocaleManager.loc("离线激活"))
                }
                .font(.caption)
                .foregroundColor(.secondary)
            }
            .buttonStyle(.plain)

            if showOfflineInfo {
                VStack(alignment: .leading, spacing: 4) {
                    Text(LocaleManager.loc("离线激活方式："))
                        .font(.caption)
                        .fontWeight(.medium)
                    Text(LocaleManager.loc("1. 在联网电脑上访问 z-pulse.cn/offline"))
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Text(LocaleManager.loc("2. 输入购买邮箱和本机设备码"))
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Text(LocaleManager.loc("3. 下载授权文件并导入本程序"))
                        .font(.caption)
                        .foregroundColor(.secondary)
```

```
Button {
                        importLicenseFile()
                    } label: {
                        Label(LocaleManager.loc("导入授权文件..."), systemImage: "doc.badge.plus")
                            .frame(maxWidth: .infinity)
                    }
                    .buttonStyle(.bordered)
                    .padding(.top, 8)
                }
                .padding(12)
                .background(Color(nsColor: .controlBackgroundColor))
                .clipShape(RoundedRectangle(cornerRadius: 8))
            }

            Spacer()
        }
        .padding(.horizontal, 48)
        .frame(width: 420, height: 520)
        .background(Color(nsColor: .windowBackgroundColor))
    }

    private var validInput: Bool {
        keyInput.count == 14 // XXXX-XXXX-XXXX
    }

    private func activate() {
        guard validInput else { return }
        isActivating = true
        errorMessage = nil
        successMessage = nil

        Task {
            let result = await licenseManager.activate(key: keyInput)
            await MainActor.run {
                isActivating = false
                switch result {
                case .success:
                    licenseManager.showActivationSheet = false
                case .failure(let error):
                    errorMessage = error.localizedDescription
                }
            }
        }
    }

    private func importLicenseFile() {
        errorMessage = nil
        successMessage = nil

        guard let result = licenseManager.importOfflineLicenseFile() else { return }
        switch result {
        case .success(let plan, _):
            successMessage = LocaleManager.loc("授权文件已导入：%@").replacingOccurrences(of: "%@", with: plan.displayName)
            licenseManager.showActivationSheet = false
        case .failure(let error):
            errorMessage = error.localizedDescription
        }
    }
}
```

```
// MARK: - License Status Badge

/// Small indicator showing license status in toolbar
public struct LicenseStatusBadge: View {
    @ObservedObject var licenseManager: LicenseManager

    public init(licenseManager: LicenseManager = .shared) {
        self.licenseManager = licenseManager
    }

    public var body: some View {
        HStack(spacing: 4) {
            Circle()
                .fill(color)
                .frame(width: 6, height: 6)
            Text(label)
                .font(.caption)
                .foregroundColor(.secondary)
        }
    }

    private var color: Color {
        switch licenseManager.licenseState {
        case .activated:   return .green
        case .expired,
             .unactivated:  return .red
        case .gracePeriod: return .orange
        case .trial:       return .blue
        }
    }

    private var label: String {
        switch licenseManager.licenseState {
        case .activated:   return licenseManager.plan.displayName
        case .unactivated: return LocaleManager.loc("未激活")
        case .expired:     return LocaleManager.loc("已过期")
        case .gracePeriod: return LocaleManager.loc("离线模式")
        case .trial(let remaining): return String(format: LocaleManager.loc("试用 %lld 天"), remaining)
        }
    }
}
```